

Table des matières

Introduction	3
Architecture.....	4
Partie 1 : Acquisition et encodage	5
1.1 Récupération des Données :	5
1.2 Fonction affichage matrice LEDS :.....	5
1.3 CayenneLPP :.....	6
Partie 2 : Transmission des Données	8
2.1 Configuration du Module LoRa	8
2.1 - Transmission de Données :.....	9
2.1.1 - fonction d'envoi de données :	9
2.1.2 Application :.....	10
2.1.3 Vérification d'envoi de données :	10
3.1 Architecture node-red:	12
3.2 Interface de visualisation.....	12
Partie 4 : Analyse de Performance	14
4.1 Récupération de trames	14
4.2 Analyse.....	14
4.2.1 Variation des valeurs RSSI en fonction du SF	14
4.2.2 Variation des valeurs SNR en fonction du SF	15
4.2.3 Data Extraction Rate de chaque Gateway pour chaque SF	16
4.2.4 Impact de la Distance sur le RSSI, le SNR et le Choix du SF	17
Conclusion	18

Introduction

Dans un contexte marqué par une avancée technologique rapide, **l'Internet des Objets (IoT)** joue un rôle central dans la transformation numérique mondiale, grâce à sa capacité à connecter des dispositifs et à exploiter les données en temps réel.

Notre projet, initié en octobre 2024, s'inscrit dans cette dynamique et vise à concevoir un système IoT complet, intégrant les étapes essentielles : acquisition des données, transmission et visualisation intuitive.

Les données environnementales, telles que la température, l'humidité et la pression, sont captées à l'aide d'un Sense HAT connecté à un Raspberry Pi. Ces données sont encodées selon le protocole **Cayenne LPP**, garantissant une interopérabilité accrue tout en minimisant la taille des messages par rapport aux formats classiques comme JSON.

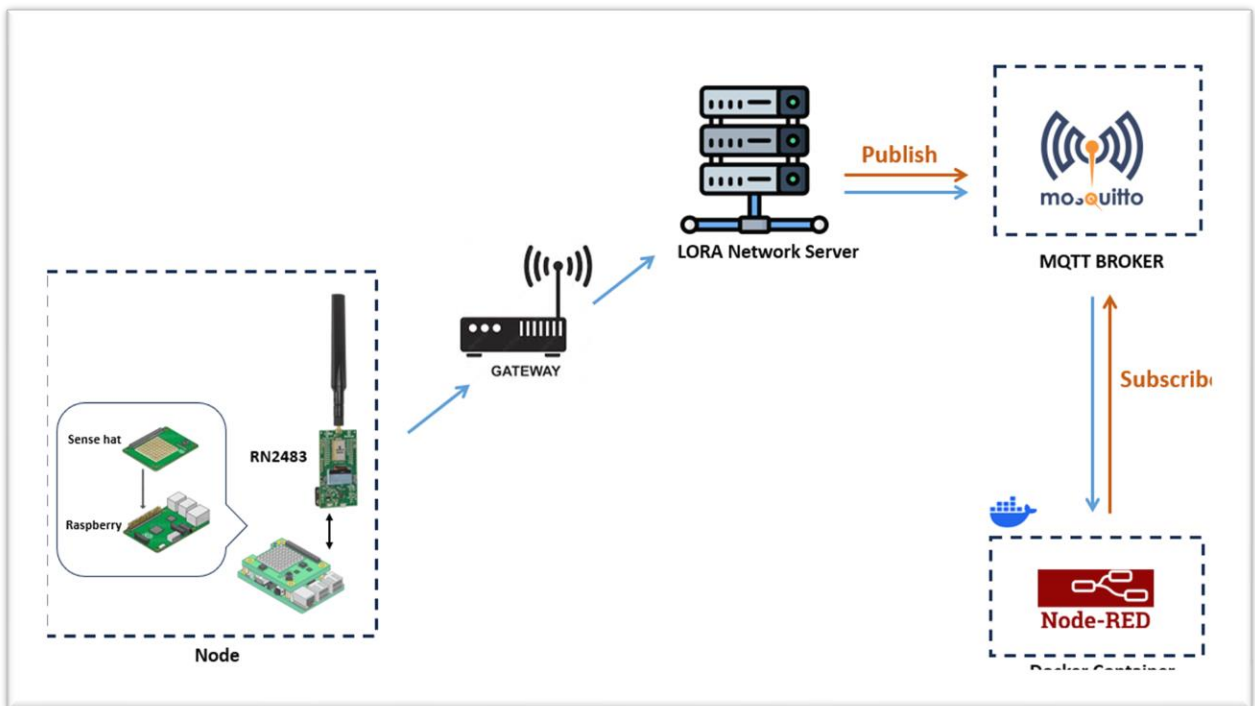
Pour leur transmission, nous utilisons la technologie LoRaWAN, réputée pour sa faible consommation énergétique et sa portée étendue. Ces données transitent via une gateway **LoRaWAN** connectée au réseau de l'université neOCampus. Une fois reçues par le serveur LoRa, elles sont publiées sur des topics spécifiques grâce à un broker **MQTT** (Mosquitto), utilisant un protocole léger et optimisé pour la communication des objets connectés.

Les données reçues sont ensuite décodées et affichées sur une interface web intuitive conçue avec Node-RED, permettant une consultation et une exploitation simplifiées des mesures collectées.

Dans le cadre de ce projet, une étude approfondie a été réalisée sur les **performances** du protocole LoRaWAN. Cette analyse s'est concentrée sur le paramètre clé du facteur d'étalement (Spreading Factor, SF). En ajustant ce paramètre, nous avons capturé et analysé les trames en fonction d'indicateurs comme le RSSI et le SNR, générant ainsi des graphiques pour mieux comprendre l'impact du SF sur les performances du système.

Ce projet illustre une approche méthodique et rigoureuse, contribuant à l'optimisation des solutions IoT et renforçant notre compréhension des technologies de communication sans fil.

Architecture



Architecture globale du projet

Partie 1 : Acquisition et encodage

1.1 Récupération des Données :

Afin de récupérer les données environnementales telles que la température , humidité , pression nous avons utilisé sense hat qui est un module qui offre à Raspberry une gamme de capacité de détection par des capteurs embarqués.

comme vous pouvez voir dans ce petit programme python qui se base sur la bibliothèque SenseHat:

```
#on cherche l'humidité
humidity = sense.get_humidity()
#on cherche la temperature
temperature = sense.get_temperature()
#on cherche la pression*
pressure = sense.get_pressure()
#on cherche le gyroscope
gyroscope = sense.get_gyroscope()
#affichage des données :
print(f"Humidity: {humidity} , Temperature : {temperature} , pressure :{pressure},gyroscope={gyroscope}")
```

Résultats :

```
Humidity: 49.67790985107422 , Temperature : 31.756998825073242 , pressure :1009.581787109375,gyroscope={'roll': 0.0, 'pitch': 0.0, 'yaw': 0.0}
```

1.2 Fonction affichage matrice LEDS :

Nous souhaitons également visualiser les données collectées sur la matrice LED du SenseHat, en complément de leur affichage dans le terminal.

Pour cela, nous avons créé une fonction qui prend en paramètre la donnée souhaitée puis la collecte et l'affiche sur la matrice LED illustrée dans la figure de résultat :

Dans cet exemple on a choisi d'afficher la **température** :

```
def afficher_mesure (mesure):
    """
    fonction pour afficher une mesure souhaitée dans la matrice du module sense Hat
    """
    value=""
    match mesure:
        case "humidity" :
            #on cherche l'humidité
            value = sense.get_humidity()
        case "temperature":
            #on cherche la temperature
            value = "t="+round(sense.get_temperature(),1)
        case "pressure":
            #on cherche la pression*
            value = sense.get_pressure()
        case "gyroscope":
            #on cherche le gyroscope
            value = sense.get_gyroscope()
    # affichage sur la matrice des LEDS

    sense.show_message(str(value))
    #test pour affichage de mesure temperature : afficher_mesure ("temperature")

afficher_mesure ("temperature")
```

Résultat :



La valeur de la température du capteur s'affiche avec succès sur la matrice LED. Plus précisément, les données défilent, offrant une visualisation claire des valeurs présentées dans le terminal de la Raspberry pi.

1.3 CayenneLPP :

Nous avons implémenté une fonction d'encodage des données selon le format CayenneLPP, spécialement conçu pour optimiser la payload contenant les données d'un ou plusieurs capteurs, afin de réduire la consommation de batterie, qui représente l'une des principales limitations en IoT.

Voici un exemple de code Python qui génère l'encodage des données au format **CayenneLPP** pour des capteurs comme la température, l'humidité :

```
try:
    from cayennelpp import LppFrame
except:
    !pip install pycayennelpp
    from cayennelpp import LppFrame

#create frame
frame = LppFrame()
# add some sensor data
#température
frame.add_temperature(0, round(sense.get_temperature(),1))
#humidité
frame.add_humidity(6, sense.get_humidity() )
#creation du buffer
buffer = bytes(frame)
print(buffer)

b'\x00g\x010\x06he'
```

Le format CayenneLPP est plus adapté aux applications IoT que d'autre format comme JSON pour plusieurs avantages ,parmi eux :

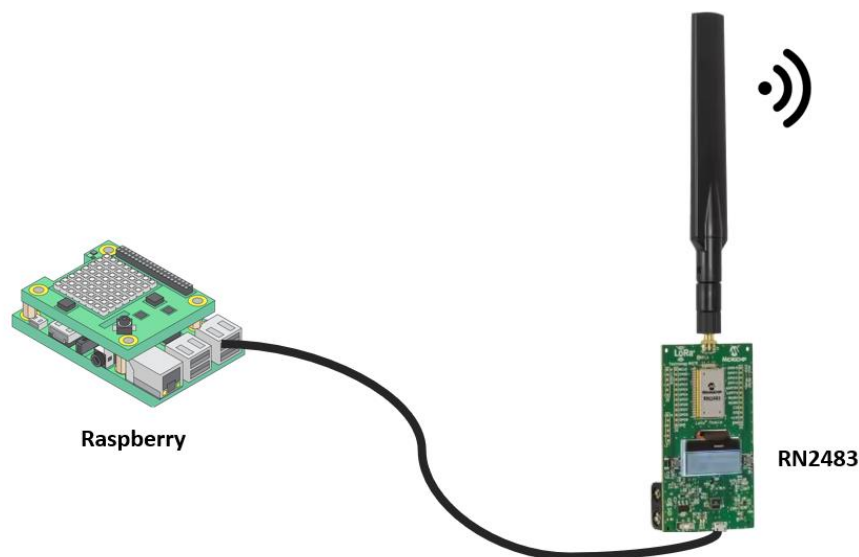
1. **Taille optimisée** : Grâce à son encodage binaire compact, CayenneLPP réduit considérablement la taille des données par rapport à des formats comme JSON. Par exemple, un message JSON de 40 octets peut être compressé en quelques octets seulement avec CayenneLPP.
2. **Adapté aux réseaux LoRaWAN** : Spécialement conçu pour les réseaux à faible débit, permettant d'envoyer davantage de données ou d'augmenter la fréquence des transmissions tout en respectant les contraintes de durée d'émission (Time on Air).
3. **Économie d'énergie** : La réduction de la taille des messages diminue le temps de transmission, ce qui contribue à économiser la batterie des dispositifs IoT, souvent utilisés sur le long terme.
4. **Compatibilité standardisée** : CayenneLPP est un standard largement adopté dans l'IoT, offrant une interopérabilité accrue avec les plateformes compatibles LoRaWAN.

En résumé, CayenneLPP offre une solution plus légère et économe pour transmettre les données entre les dispositifs IoT grâce à son efficacité énergétique et sa compacité.

Partie 2 : Transmission des Données

2.1 Configuration du Module LoRa

Pour cette partie, nous avons transmis les données captées et encodées vers les serveurs applicatifs en utilisant la technologie LoRaWAN avec l'utilisation d'un module LoRa Mote de microchip (RN2483) .



La communication entre le **Raspberry Pi** et le **module LoRa Mote RN2483** s'effectue via une liaison série (UART).

Pour établir cette communication, une configuration est utilisée pour définir le port et le débit de transmission série.

Pour pouvoir utiliser efficacement le module nous devons le configurer par les paramètres et les identifiants LoRaWAN de notre application afin que nos données soient bien transmises et avec les bons paramètres.

ce processus est divisé sur plusieurs opérations :

1. **Activation OTAA (appkey):**
 - Commande : **mac set appkey + appkey**
2. Configuration de l'Identifiant d'Application (**appEUI**) :
Cela spécifie l'application à laquelle le module LoRa doit se connecter.
 - Commande : **mac set appeui + appEui**
3. Configuration de l'Identifiant du Dispositif (**devEUI**) :
 - Commande : **mac set deveui + deveui**
4. Configuration de facteur d'étalement (**Spreading Factor**) :
 - Commande : **mac set dr + datarate**

pour notre module la configuration de SP est proportionnelle avec Data Rate comme présenté dans ce tableau :

Data Rate	Spreading Factor	Bandwidth	bits/s
DR0	SF12	125 kHz	250
DR1	SF11	125 kHz	440
DR2	SF10	125 kHz	980
DR3	SF9	125 kHz	1760
DR4	SF8	125 kHz	3125
DR5	SF7	125 kHz	5470
DR6	SF7	250 kHz	11000
DR7	FSK	50 kbps	50000

Ce facteur contrôle l'étalement du chirp dans le temps. Plus le Spreading Factor (SF) est élevé, plus le débit diminue, mais la portée et la résistance au bruit augmentent. À l'inverse, un SF plus faible entraîne une augmentation du débit, une diminution de la portée et une résistance réduite au bruit.

5. Sauvegarde des configurations du module
commande : **mac save**

Voici nos paramètres et identifiants LoRaWAN utilisés pour ce projet :

```
appkey = "0123456789ABCDEF0123456789ABCDEF"  
appeui = "DEAD25DEAD25DEAD"  
deveui = "DEADDEAD00090009"
```

1. **appkey** = "0123456789ABCDEF0123456789ABCDEF"
2. **appeui** = "DEAD25DEAD25DEAD"
3. **deveui** = "DEADDEAD00090009"

2.1 - Transmission de Données :

2.1.1 - fonction d'envoi de données :

Après la Configuration de notre module LoRa c'est le temps pour envoyer les messages au **Lora Network Server**.

Pour cela nous avons développé cette fonction :

```
def send_message(sp,message:hex):  
    #Send the command to send the message  
    command = "mac tx cnf 5 "+str(message)  
    response = send(sp,command)  
    return response
```

Cette dernière envoie tout simplement une commande d'envoi au module LoRa avec le message que nous souhaitons envoyer .

La commande “**mac tx cnf**” signifie que le module LoRa effectue une transmission confirmée c'est-à- dire il attendra un accusé de réception (ACK) du serveur LoRaWAN pour garantir la bonne réception du message.

2.1.2 Application :

Cette application a pour objectif d'envoyer les données environnementales captées, tout en tirant parti de ce que nous avons développé auparavant (mesure des données, encodage, configuration du module et envoi).

Il envoie **la température, l'humidité et la pression** une fois chaque 5 minutes.

```
from sense_hat import SenseHat
from cayennelpp import LppFrame
import time

#Fonction qui retourne les donnees de temperature, d'humidite et de pression encodees au format CayenneLPP
def get_sensor_data():
    sense = SenseHat()
    sense.clear()
    #create frame
    frame = LppFrame()
    # add some sensor data
    #1-temperature
    frame.add_temperature(0, round(sense.get_temperature(),1))
    # 2-Humidity
    frame.add_humidity(6, sense.get_humidity())
    # 3-Pression
    frame.add_pressure(3,sense.get_pressure())
    #creation du buffer
    buffer = bytes(frame)
    return buffer.hex()

#Fonction qui definit la periode de mesure et d'envoi des donnees
def periodic_send_data(sp):
    while(1):
        send_message(sp,get_sensor_data())
        #periode = 5 minutes (300s)
        time.sleep(300)

def start_application():
    #Configuration de module LoRa
    response = config_module(module,"0123456789ABCDEF0123456789ABCDEF","DEAD25DEAD25DEAD","DEADDEAD00090009",12)
    logging.info("Did we join the network ? %s",response)
    #Mesures et Envoi periodiques
    periodic_send_data(module)
```

2.1.3 Vérification d'envoi de données :

On sait que le **LoRa Network Server (LNS)** est configuré pour publier les messages reçus sur le topic suivant : **TestTopic/lora/neoCampus/{deveui}/{event}**.

Pour vérifier que nos messages sont bien reçus par le LNS, une simple commande en ligne est exécutée pour s'abonner à ce topic et observer les messages envoyés. Cela permet de confirmer la réception correcte des données transmises.

la commande:

```
mosquitto_sub -h neocampus.univ-tlse3.fr -t TestTopic/lora/neoCompus/DEADDEAD00090009/# -p 1882 -u test -P test
```

Résultats :

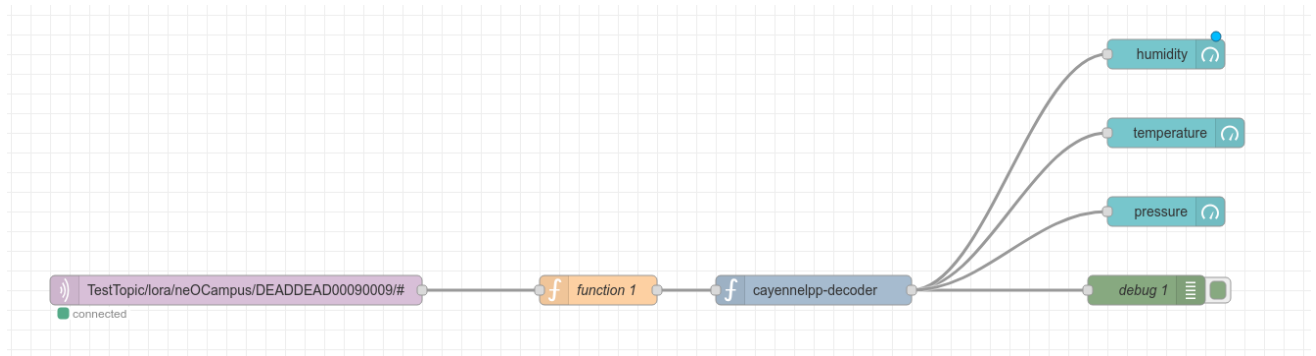
```
{
  "desc": "Lab-device-09",
  "devaddr": "85ADAEBE",
  "deveui": "DEADDEAD00090009",
  "fcnt": 0,
  "freq": 867.7,
  "lsnr": -1.5,
  "mac": "008000000000C2F1",
  "all_gw": [
    {
      "desc": "MTCOTIP-266A / LoRa gateway @ Ecolab",
      "gpsalt": 167,
      "gpspos": {
        "lat": 43.55804,
        "lon": 1.46955
      },
      "lsnr": 5.2,
      "mac": "0000000000000000",
      "port": 220,
      "rssi": -101,
      "type": "T2-460",
      "gpsalt": 155,
      "gpspos": {
        "lat": 43.56223260262245,
        "lon": 1.4676833152770996
      },
      "lsnr": 7.5,
      "mac": "FFFE000448EC61C2",
      "rssi": -104,
      "type": "T2-460",
      "port": 255,
      "best_gw": {
        "desc": "MTCOTIP-266A / LoRa gateway @ Ecolab",
        "gpsalt": 167,
        "gpspos": {
          "lat": 43.55804,
          "lon": 1.46955
        },
        "lsnr": 5.2,
        "mac": "0000000000000000",
        "port": 220,
        "rssi": -101,
        "type": "T2-460",
        "gpsalt": 155,
        "gpspos": {
          "lat": 43.56223260262245,
          "lon": 1.4676833152770996
        },
        "lsnr": 7.5,
        "mac": "FFFE000448EC61C2",
        "rssi": -104,
        "type": "T2-460",
        "port": 255
      }
    }
  ],
  "datetime": "2024-12-04T17:57:41Z",
  "datr": "SF12BW125",
  "desc": "Lab-device-09",
  "devaddr": "85ADAEBE",
  "deveui": "DEADDEAD00090009",
  "freq": 867.7,
  "lsnr": -1.5,
  "mac": "008000000000C2F1",
  "port": 220,
  "rssi": -101,
  "type": "T2-460",
  "gpsalt": 167,
  "gpspos": {
    "lat": 43.55804,
    "lon": 1.46955
  },
  "lsnr": 5.2,
  "mac": "0000000000000000",
  "port": 220,
  "rssi": -101,
  "type": "T2-460",
  "gpsalt": 155,
  "gpspos": {
    "lat": 43.56223260262245,
    "lon": 1.4676833152770996
  },
  "lsnr": 7.5,
  "mac": "FFFE000448EC61C2",
  "rssi": -104,
  "type": "T2-460",
  "port": 255
}
```

Comme le montrent les résultats, nous pouvons confirmer que nos données parviennent correctement à leur destination.

Partie 3 : Décodage et Visualisation

3.1 Architecture Node-Red:

L'architecture Node-RED mise en place traite les messages MQTT reçus sur notre topic, les décode et les distribue sous forme de données pour l'humidité, la température et la pression, permettant par la suite leur affichage sur des tableaux de bord.



Pour le décodage des messages au format **CayenneLPP**, nous avons utilisé une fonction de décodage fournie, appelée **cayennelpp-decoder**. Cependant, cette fonction accepte uniquement la partie **data** de la payload du message original envoyé. C'est pourquoi nous avons mis en place une autre fonction (**function 1**) qui extrait cette partie spécifique de la payload avant de procéder au décodage.

code function 1 :

```
msg.payload=msg.payload.data  
return msg;
```

remarque : le développement de la fonction est en javascript.

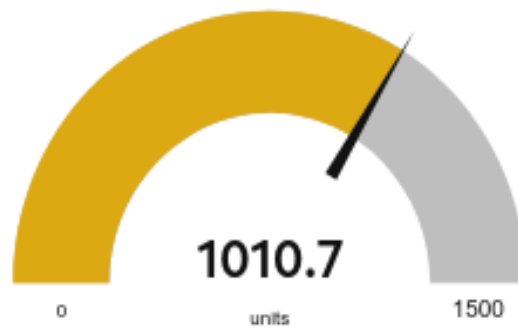
3.2 Interface de visualisation

Grâce au module **node-red-dashboard**, nous pouvons visualiser les données sur une interface conviviale de tableau de bord.

Voici les résultats reçus :

dashboard

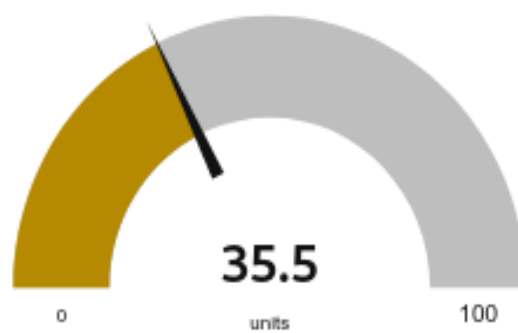
pressure



temperature



humidity



Partie 4 : Analyse de Performance

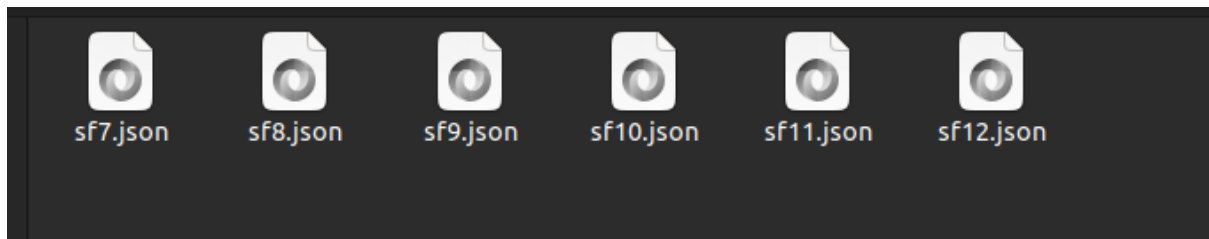
Cette partie de TP a pour objectif d'étudier l'impact du facteur d'Étalement (Spreading Factor ou SF) sur les performances du protocole LoRaWAN. Les objectifs principaux sont :

1. Comprendre comment le SF influence la portée, le débit et la robustesse des transmissions.
2. Collecter et analyser des données en modifiant les paramètres de SF.
3. Visualiser les performances à l'aide de graphiques.

4.1 Récupération de trames

Dans cette partie, nous avons récupéré les trames manuellement et les avons enregistrées dans des fichiers JSON distincts pour chaque **Spreading Factor** (SF 7, 8, 9, 10, 11, 12). Pour chaque SF, un total de 30 trames a été collecté.

Ces fichiers serviront de base pour l'analyse des performances dans la prochaine partie.

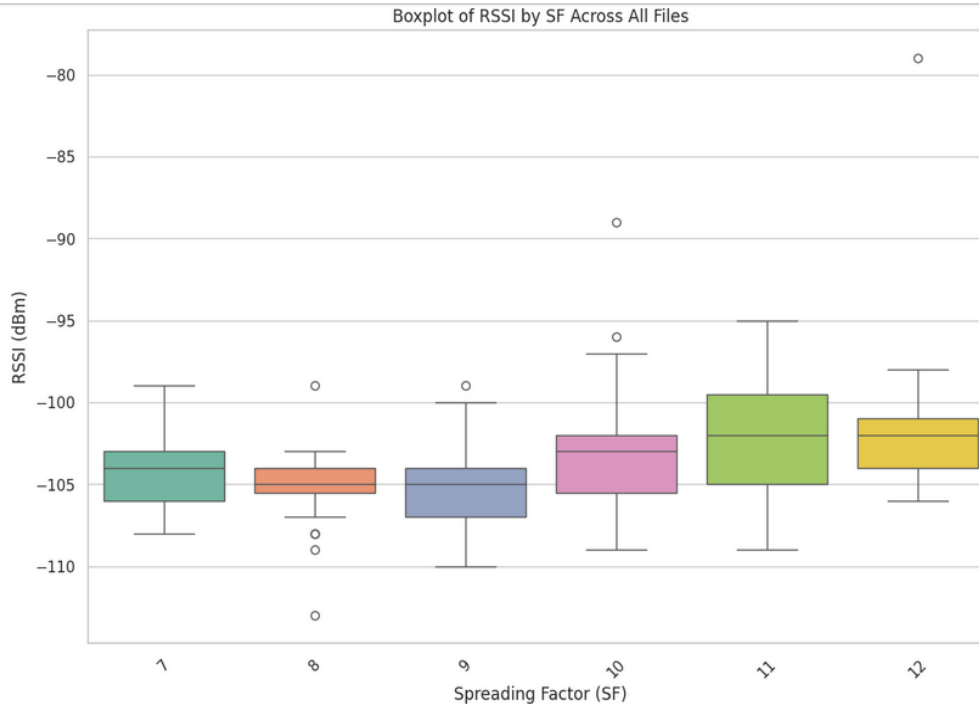


4.2 Analyse

Dans cette partie, nous réutilisons les trames enregistrées lors de la partie 1. Lors du traitement des trames, nous nous concentrons sur les valeurs de la best gateway.

4.2.1 Variation des valeurs RSSI en fonction du SF

Le graphique ci-dessous illustre les variations des valeurs RSSI en fonction du Spreading Factor (SF). Ces résultats mettent en évidence l'impact du SF sur la puissance du signal reçu, permettant d'analyser les performances des transmissions selon les conditions de propagation.

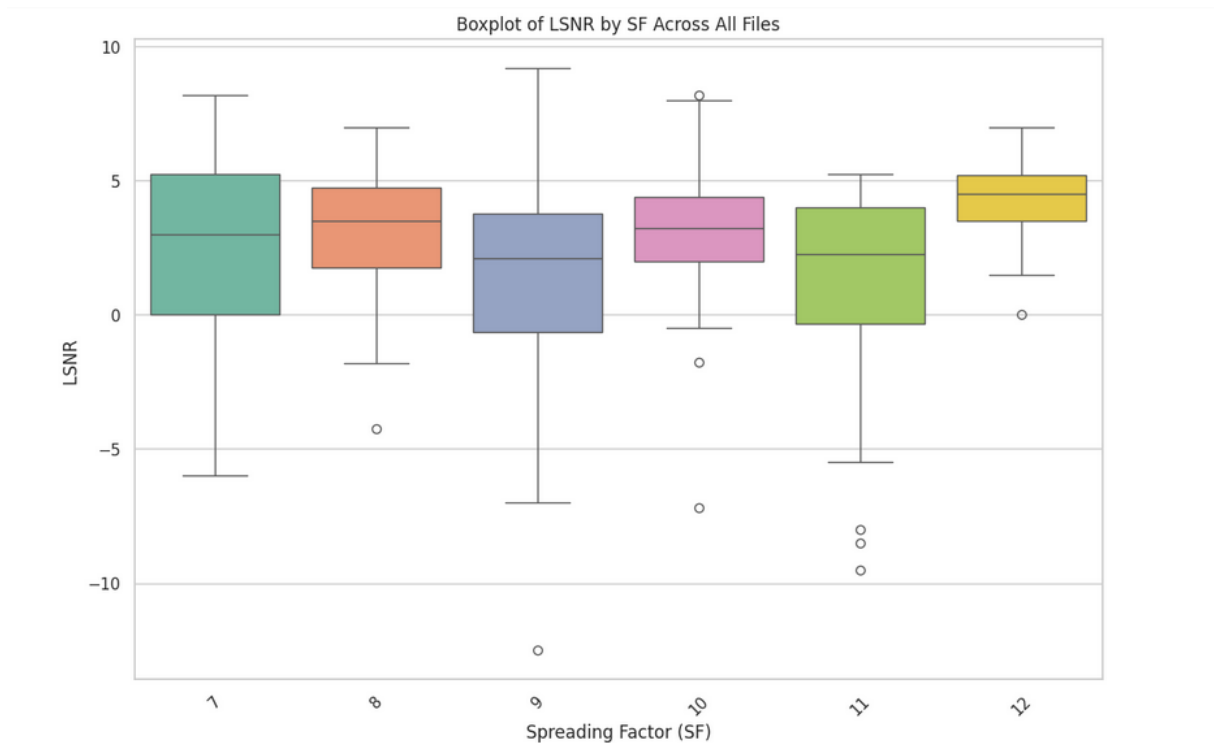


Les valeurs de RSSI (exprimées en dBm) montrent une certaine variabilité pour chaque Spreading Factor (SF). On observe que le RSSI tend à légèrement augmenter avec le SF, ce qui est conforme aux attentes théoriques. Cependant, l'écart constaté entre les différents SF est nettement plus faible que prévu, ce qui limite la possibilité d'observer un effet marqué de l'augmentation du SF sur la puissance du signal reçu.

Cette stabilité relative des résultats peut s'expliquer par le fait que les mesures proviennent de la meilleure gateway (best gateway), généralement la plus proche du module de transmission. Ainsi, malgré l'augmentation du SF, les gateways sélectionnées captent des signaux similaires en termes de puissance. De plus, la configuration des tests, notamment la distance constante et les conditions homogènes, pourrait également jouer un rôle dans la réduction des variations observées.

4.2.2 Variation des valeurs SNR en fonction du SF

Les variations des valeurs de SNR en fonction du Spreading Factor (SF) ont été analysées à partir des données collectées. Le graphique ci-dessous illustre ces variations pour chaque SF, en mettant en évidence les tendances générales et la distribution des valeurs observées.



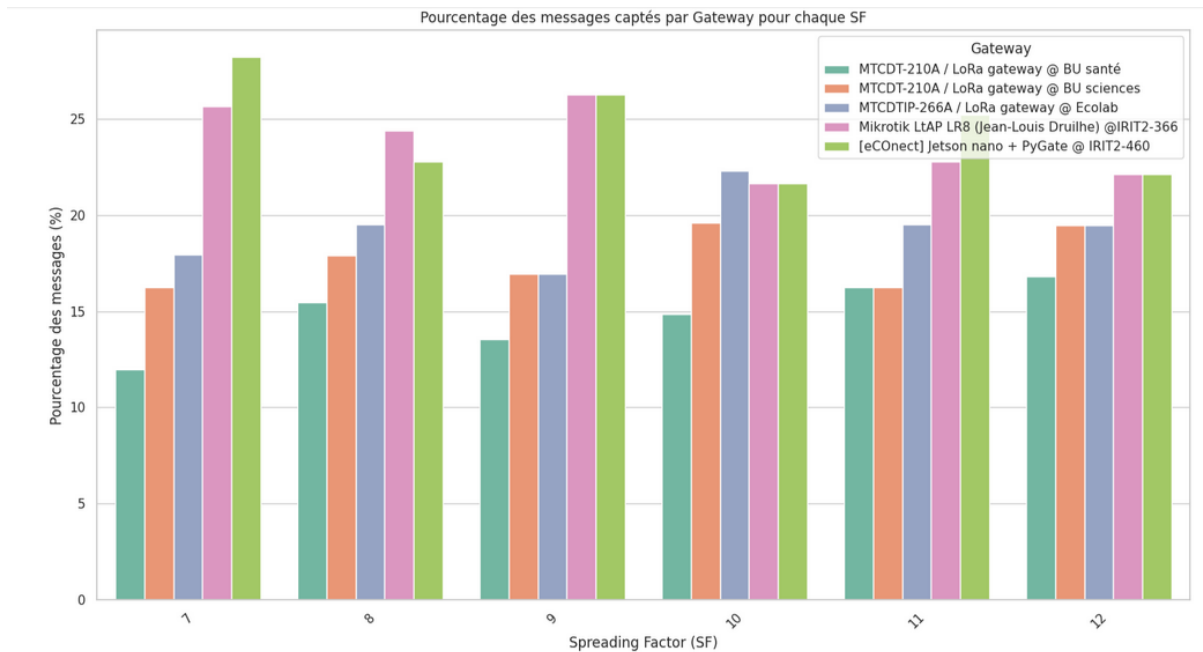
Les résultats montrent que les valeurs de SNR (*Signal-to-Noise Ratio*), exprimées en dB, ne varient pas de manière marquée en fonction du Spreading Factor (SF), contrairement à ce qui était initialement attendu. Théoriquement, on s'attend à ce que le SNR diminue avec l'augmentation du SF, car un SF plus élevé est conçu pour fonctionner dans des environnements plus bruités afin de favoriser la portée. Cependant, cette tendance n'est pas clairement observée dans les résultats obtenus.

Cette stabilité relative des valeurs de SNR peut s'expliquer par le fait que les données proviennent de la meilleure gateway (*best gateway*), qui est souvent la plus proche du module de transmission. Cela signifie que les signaux reçus sont déjà optimisés, et les effets des variations de SF sont atténués par la proximité de la gateway sélectionnée. Ainsi, même si le SF varie, les gateways captent des signaux relativement stables en termes de qualité.

4.2.3 Data Extraction Rate de chaque Gateway pour chaque SF

Dans cette partie, nous analysons le pourcentage de messages captés par chaque gateway en fonction du Spreading Factor (SF). Cette analyse permet de comprendre comment les différents gateways contribuent à la réception des données selon les variations du SF.

Le graphique ci-dessous présente la répartition en pourcentage des messages captés par chaque gateway pour chaque Spreading Factor (SF). Cette analyse permet de visualiser l'impact des différents SF sur la performance des gateways en termes de réception des messages.



L'analyse du Data Extraction Rate (DER) montre une répartition différente des messages captés par chaque gateway selon le Spreading Factor (SF). Avec un SF bas, les gateways proches, comme [eCOnect] Jetson nano + PyGate @ IRT2-460 (5 m) et Mikrotik LtAP LR8 @ IRT2-366 (150 m), captent la majorité des messages. Cela s'explique par leur proximité et leur meilleure capacité à recevoir des signaux avec un faible SF.

En revanche, un SF moyen ou élevé favorise une meilleure répartition des messages entre toutes les gateways, permettant aux gateways plus éloignées, comme MTCDDTIP-266A @ Ecolab (500 m) ou MTCDDT-210A @ BU santé (1200 m), de capter une part plus importante des transmissions. Ce comportement est attendu, car un SF élevé privilégie la portée, rendant les signaux plus accessibles aux gateways situées à grande distance.

4.2.4 Impact de la Distance sur le RSSI, le SNR et le Choix du SF

La distance joue un rôle important dans les transmissions LoRaWAN, notamment sur le RSSI, le SNR et le choix du Spreading Factor (SF).

Impact sur le RSSI et le SNR : À mesure que la distance entre l'émetteur et la gateway augmente, le RSSI diminue (signal plus faible) et le SNR baisse (rapport signal/bruit réduit). Cela s'explique par l'atténuation du signal sur de longues distances et par l'influence croissante des interférences.

Rôle dans le choix du SF : Un SF faible est adapté aux courtes distances, car il maximise le débit de transmission. À l'inverse, un SF élevé est nécessaire pour couvrir de longues distances, car il augmente la portée et garantit une meilleure fiabilité des transmissions, même avec un RSSI et un SNR plus faibles.

En conclusion, la distance est un facteur déterminant pour ajuster les paramètres de transmission LoRaWAN afin d'équilibrer portée, débit et qualité de réception.

Conclusion

En conclusion, ce projet a permis de mettre en œuvre de manière concrète des technologies essentielles à l'Internet des Objets, telles que LoRaWAN et MQTT, en intégrant des outils novateurs comme Node-RED.

À travers les différentes étapes (acquisition, encodage, transmission et visualisation des données) nous avons démontré l'efficacité de ces solutions et renforcé nos compétences en traitement et analyse de données IoT.

Malgré les défis, comme les contraintes de transmission et les limitations des infrastructures, nous avons su adopter une approche rigoureuse et méthodique pour les surmonter. Les analyses effectuées, notamment sur les performances du facteur d'étalement (SF), ont apporté des résultats concluants, consolidant notre compréhension des protocoles de communication longue portée.

Ce travail constitue une expérience formatrice, nous préparant à relever des défis similaires dans un cadre professionnel et à contribuer activement à l'évolution des technologies IoT.